

KNOWING WHEN TO THINK: A FROZEN-BACKBONE DECISION HEAD FOR ROUTING BETWEEN FAST AND SLOW TYPED DE- CISIONS

Anonymous authors

Paper under double-blind review

ABSTRACT

System One decision models answer typed questions (a choice among options, a score on a scale, a yes/no) in a single forward pass by reading option logits at an answer boundary instead of generating text. They are fast and cheap, but they cannot reason, and on hard questions they are wrong. We ask whether such a model can decide, at no extra token cost, which questions are worth the price of slow thinking. We attach a small *decision head* beside the language-model head at the readout position of a frozen backbone. The head reads the same hidden state that produces the fast answer and predicts the joint outcome of the fast path and of the backbone’s own thinking mode: whether each would be right or wrong. A question is escalated when the predicted accuracy gain from thinking exceeds a cost λ chosen at request time. Labels for the head are produced automatically by running both paths once on a labelled set; no human annotation of “should think” is needed, and no backbone weight is modified. On MMLU-Pro with a 4B-parameter Qwen backbone, fast-only accuracy is 0.540 and always-thinking accuracy is 0.740 at a mean cost of 994 generated tokens per question. Routing with the decision head reaches 0.717 while thinking on 53% of questions, recovering 89% of the accuracy gain for 53% of the token cost, and beats a router that sees only the fast answer distribution at every escalation rate. The routing signal is carried by an intermediate layer’s hidden state passed through a whitened low-rank projection; raw hidden states overfit at this data size.

1 INTRODUCTION

A growing class of deployed language-model workloads are not conversations but decisions: route this ticket, is this claim supported, how urgent is this state, which of these five actions next. Typed decision models such as Jev (TypeSafe AI, 2026) serve exactly this shape. The caller supplies a state and a set of typed questions; the model returns, for each question, an option key, a probability distribution over the options, and a confidence, without generating a single answer token. The mechanism is old (Hendrycks & Gimpel, 2017): prefill the state, append each question and its options, and read the logits of the option codes at an answer boundary. What is new is treating this readout as the product, with a request/response contract that code can consume directly, and the resulting latency and cost, one to two orders of magnitude below generate-then-parse.

The price of the single pass is that the model cannot think. On questions that require multi-step reasoning the fast readout is a guess, and on MMLU-Pro (Wang et al., 2024) a 4B backbone’s fast readout is right 54% of the time while the same backbone with its thinking mode enabled is right 74% of the time. Always thinking closes the gap but costs a thousand generated tokens per question, which is the cost the fast path exists to avoid. The interesting regime is in between: think only where thinking pays.

This paper asks whether the model can make that call itself, in the same forward pass that produces the fast answer, without modifying the backbone. Our answer is a *decision head* (Figure 1): a small multilayer perceptron attached beside `lm_head` at the readout position. It consumes the hidden

state that the language-model head already reads, plus a few statistics of the fast answer distribution, and predicts a four-way outcome: whether the fast answer is right and whether the slow answer would be right. Escalation follows from a one-line rule, escalate when $P(\text{fast wrong, slow right}) - P(\text{fast right, slow wrong}) > \lambda$. The threshold λ is the per-question price of thinking and is set at request time, so a deployment moves along the cost–accuracy curve without retraining.

Three properties make this practical. First, **the backbone is frozen and its behaviour is unchanged**. The fast path is exactly the original decision model; the slow path is exactly the backbone’s native thinking mode; the head only decides which one answers. Second, **labels are free**. Because the slow path is deterministic given the frozen backbone, running both paths once on a labelled question set yields the four-way outcome label for every question. Third, **the decision costs no tokens**. The head reads a hidden state that is computed anyway.

We evaluate on MMLU-Pro with Qwen3.5-4B, holding out entire subject categories so that reported numbers measure transfer to unseen domains. Our contributions are:

- A frozen-backbone architecture for self-routed fast/slow typed decisions, with a request-time cost knob and an unchanged fast-path contract (Section 3).
- A self-labelling procedure and a training recipe, grouped cross-validation over subject categories with a whitened low-rank bottleneck on the hidden state, that avoids the overfitting we observed with raw hidden states (Section 3.5).
- Evidence that the routing signal lives in an intermediate layer’s hidden state and is not recoverable from the fast answer distribution alone: at matched escalation rates the head beats a distribution-only router by about three accuracy points, and recovers 89% of the always-think gain at 53% of its cost (Section 4).

We also report what did not work, including a prompt-template interaction that silently disabled thinking, because these failure modes are likely to recur for anyone building on hybrid-thinking checkpoints.

2 RELATED WORK

Typed decision models and constrained readout. Reading a classifier off a language model’s next-token distribution over a small label set is the standard zero-shot protocol for multiple-choice evaluation (Hendrycks et al., 2021; Wang et al., 2024). Jev (TypeSafe AI, 2026) productised this readout as a typed decision API with parallel isolated questions and zero generated output tokens; a wave of open reproductions followed, including prefix-cached implementations on stock checkpoints (Zhang, 2026; zhihz, 2026) and small trained scorers (Mapika, 2026). Our fast path is one such reproduction. None of these systems can think; we add the ability to decide when to.

Adaptive computation and thinking budgets. Adaptive computation time (Graves, 2016) and confident early exit (Schuster et al., 2022) allocate depth per token. Test-time scaling work (Snell et al., 2024; Muennighoff et al., 2025) shows that thinking length is a first-class compute knob, and AdaptThink (Zhang et al., 2025) and L1 (Aggarwal & Welleck, 2025) train the backbone with reinforcement learning to choose or bound its own reasoning length. We differ in leaving the backbone untouched: the routing policy is an external probe, trained by supervised learning on automatically produced labels, and the backbone’s fast and slow behaviours are exactly what it shipped with.

Model routing. FrugalGPT (Chen et al., 2023) and RouteLLM (Ong et al., 2024) route queries between cheap and expensive *models* using a learned router over the query text. We route between two *modes of the same model*, and the router reads the model’s own internal state rather than the input, which is what allows it to cost nothing at inference: the state is already computed.

Confidence estimation from hidden states. Maximum softmax probability is a strong baseline for detecting misclassification (Hendrycks & Gimpel, 2017), and selective classification formalises the coverage–accuracy trade-off we plot (Geifman & El-Yaniv, 2017). Language models carry information about their own correctness in their activations (Kadavath et al., 2022; Azaria & Mitchell, 2023), recoverable with linear probes (Alain & Bengio, 2017). We use the same observation for a

different target: not “is this answer right” but “would thinking make it right”. Calibration in the sense of Guo et al. (2017) is a by-product: the head’s $P(\text{fast right})$ replaces the uncalibrated concentration statistic that the fast path would otherwise report.

3 METHOD

3.1 FAST PATH: SHARED PREFILL AND BRANCHED READOUT

A request carries a *state* s (text, or text plus an image) and N typed questions $q_1, \dots, q_N$. Each question has a type in $\{\text{choice}, \text{score}, \text{noul}\}$ and a candidate set: the option keys for *choice*, the ordered levels for *score*, and $\{\text{false}, \text{true}\}$ for *noul*. Following the typed-decision contract, question identifiers never enter the prompt and questions are isolated from one another.

The fast path uses two forward calls. The first prefills the state and a generic instruction once, producing a cache of key/value and, for hybrid backbones, recurrent states. The cache is replicated N ways. The second call batches the N question suffixes, each ending in the boundary string `Answer:`, against their cache copies. Candidate options are mapped at the boundary to single-token letter codes (A...Z, then AA...), verified per tokenizer, so that every candidate is exactly one token. At the boundary position t_i of branch i the model produces a final-layer hidden state $\mathbf{h}_i \in \mathbb{R}^d$ and, through `lm_head`, logits over the vocabulary. We slice the K_i candidate logits $\mathbf{z}_i \in \mathbb{R}^{K_i}$ and set $\mathbf{p}_i = \text{softmax}(\mathbf{z}_i/\tau)$. The typed answer is assembled in code: the `argmax` key for *choice*, the probability-weighted level index for *score*, and $p(\text{true})$ for *noul*. No token is generated.

3.2 DECISION HEAD

We attach a second reader at the same position (Figure 1). Its input is the concatenation of (i) the hidden state $\mathbf{h}_i^{(\ell)}$ at the boundary position from one chosen layer ℓ , which need not be the final layer, and (ii) an eight-dimensional statistic vector $\mathbf{s}_i$ of the fast distribution: maximum probability, normalised entropy, top-two margin, normalised Gini concentration, $\log K_i$, and a one-hot question type. The statistics are scale-free, so the head transfers across candidate-set sizes.

The head standardises its input with training-set moments, projects the hidden block through a whitened PCA basis $\mathbf{W} \in \mathbb{R}^{d \times r}$ with $r \ll d$ (Section 3.5), and applies a two-layer MLP:

$$\mathbf{u}_i = [\mathbf{W}^\top \tilde{\mathbf{h}}_i^{(\ell)}; \tilde{\mathbf{s}}_i], \quad \mathbf{o}_i = \text{softmax}(\mathbf{W}_2 \text{GELU}(\mathbf{W}_1 \mathbf{u}_i + \mathbf{b}_1) + \mathbf{b}_2) \in \Delta^4, \quad (1)$$

where tildes denote standardised quantities. The four outputs are the joint outcome probabilities

$$\mathbf{o}_i = (P(\text{fast}\checkmark, \text{slow}\checkmark), P(\text{fast}\checkmark, \text{slow}\times), P(\text{fast}\times, \text{slow}\checkmark), P(\text{fast}\times, \text{slow}\times)). \quad (2)$$

Two quantities are derived. The *confidence* reported to the caller is $c_i = \mathbf{o}_{i,1} + \mathbf{o}_{i,2} = P(\text{fast}\checkmark)$, a probability of correctness rather than a shape statistic of the distribution. The *escalation gain* is

$$g_i = \mathbf{o}_{i,3} - \mathbf{o}_{i,2} = P(\text{fast}\times, \text{slow}\checkmark) - P(\text{fast}\checkmark, \text{slow}\times), \quad (3)$$

the expected change in accuracy from switching this question to the slow path. Predicting the joint outcome rather than fast correctness alone matters: a question the fast path gets wrong is not worth escalating if the slow path would also get it wrong, and a question the fast path gets right can be harmed by thinking (4.6% of held-out questions in our data).

3.3 ROUTING RULE AND COST MODEL

Let $a_i^{\text{fast}}, a_i^{\text{slow}} \in \{0, 1\}$ be the correctness of the two paths and T_i the number of thinking tokens the slow path generates. Escalating question i changes expected accuracy by g_i and costs T_i tokens. With a per-token price the optimal policy escalates when $g_i > \lambda$ for a λ proportional to that price. We therefore expose λ directly on the request:

$$\text{escalate}_i = \mathbb{I}[g_i > \lambda]. \quad (4)$$

$\lambda = 0$ escalates whenever thinking is expected to help at all; $\lambda > 0$ demands a margin; $\lambda < 0$ escalates even at a small expected loss and approaches always-thinking. Because λ never enters

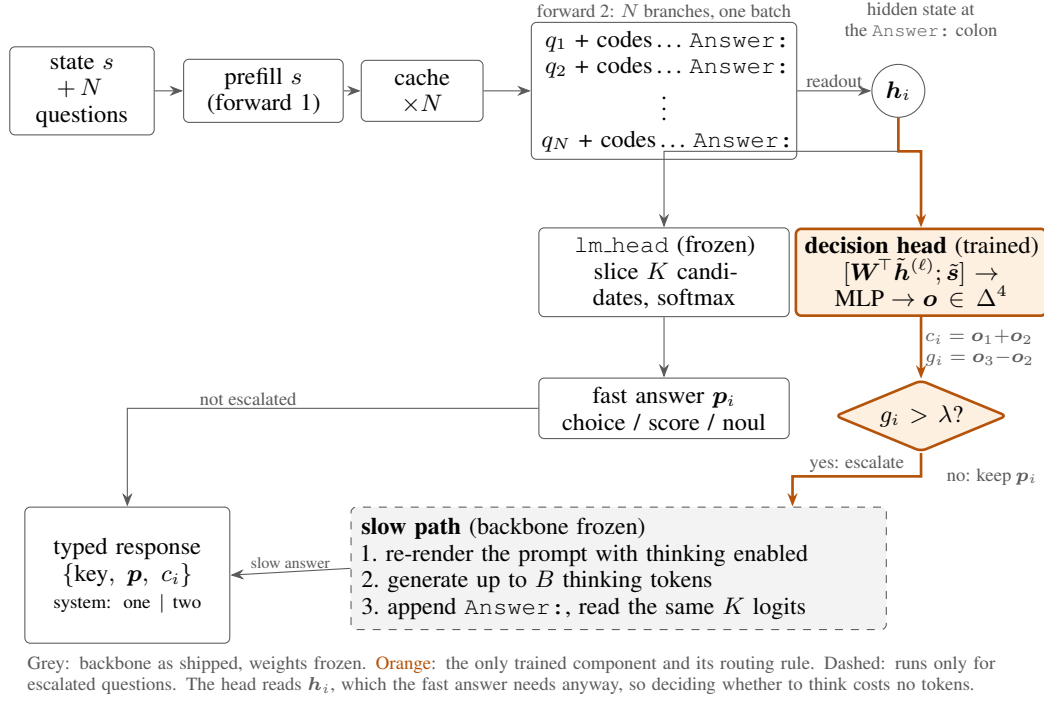


Figure 1: LitJev-S2 on one request. The state is prefilled once; N question branches share the cache and are read out in one batched forward. At the readout position the frozen `lm_head` produces the fast answer while the decision head predicts the joint (fast, slow) outcome. Questions whose escalation gain exceeds the request-time cost λ are re-run through the backbone’s own thinking mode and read out again at the same boundary.

training, a deployment sweeps the coverage–accuracy curve by changing one number. The routed accuracy at threshold λ on a set $\mathcal{D}$ is

$$\text{Acc}(\lambda) = \frac{1}{|\mathcal{D}|} \sum_{i \in \mathcal{D}} \left(\mathbb{I}[g_i > \lambda] a_i^{\text{slow}} + \mathbb{I}[g_i \leq \lambda] a_i^{\text{fast}} \right), \quad (5)$$

and the expected generated tokens per question is $\frac{1}{|\mathcal{D}|} \sum_i \mathbb{I}[g_i > \lambda] T_i$. A random router that escalates a fraction ρ of questions attains $\text{Acc} = \bar{a}^{\text{fast}} + \rho(\bar{a}^{\text{slow}} - \bar{a}^{\text{fast}})$, a straight line between the two endpoints; a useful router lies above it.

3.4 SLOW PATH

Escalated questions are re-run with the backbone’s thinking mode. Hybrid-thinking checkpoints such as Qwen3 (Yang et al., 2025) decide whether to reason from the chat template: with thinking disabled the template inserts an empty `<think>\n\n</think>` block before the assistant turn, and with thinking enabled it ends the prompt with an open `<think>`. The fast prefix is rendered with thinking disabled. Appending `<think>` to it does not work: the model has already seen a closed thinking block and closes the new one after a single token (Section 5). The slow path therefore renders its own prompt from scratch with thinking enabled, placing the question body in the user turn, generates greedily until `</think>` or a budget of B tokens, then appends the same `Answer:` boundary and reads the same K candidate logits. The typed answer is assembled exactly as on the fast path; nothing is parsed from generated text. Escalated questions are batched together for generation. The response reports `output_tokens` equal to the thinking tokens used, and marks each answer with the system that produced it.

Algorithm 1 Self-labelled data collection

Require: frozen backbone f , labelled questions $\{(s_i, q_i, y_i^*)\}$, candidate layers L , budget B

- 1: **for** each batch of ten questions with shared state s **do**
- 2: $\{(z_i, \{h_i^{(\ell)}\}_{\ell \in L})\} \leftarrow \text{FASTPATH}(f, s, \{q_i\})$ {prefill once, one branched forward}
- 3: $\{(z_i^{\text{slow}}, T_i)\} \leftarrow \text{SLOWPATH}(f, s, \{q_i\}, B)$ {thinking on, same readout boundary}
- 4: **for** each question i **do**
- 5: $a_i^{\text{fast}} \leftarrow \mathbb{1}[\arg \max z_i = y_i^*], \quad a_i^{\text{slow}} \leftarrow \mathbb{1}[\arg \max z_i^{\text{slow}} = y_i^*]$
- 6: $y_i \leftarrow 2(1 - a_i^{\text{fast}}) + (1 - a_i^{\text{slow}})$ {0: both right, 1: fast only, 2: slow only, 3: neither}
- 7: $s_i \leftarrow \text{STATS}(\text{softmax}(z_i), \text{type}(q_i))$
- 8: store $\{h_i^{(\ell)}\}, s_i, y_i, a_i^{\text{fast}}, a_i^{\text{slow}}, T_i, \text{category}(q_i)$
- 9: **end for**
- 10: **end for**

Algorithm 2 Decision head training with grouped selection

Require: records from Algorithm 1, held-out categories $\mathcal{C}_{\text{test}}$, folds K , candidate configurations Θ (layer ℓ , rank r , width w)

- 1: split records into $\mathcal{D}_{\text{train}}$ and $\mathcal{D}_{\text{test}}$ by category
- 2: assign each training category to one of K folds
- 3: **for** each configuration $\theta \in \Theta$ **do**
- 4: **for** $k = 1, \dots, K$ **do**
- 5: fit standardiser and PCA basis W_θ on folds $\neq k$; train head $H_\theta^{(k)}$ on folds $\neq k$ with early stopping
- 6: $\text{score}_k \leftarrow \text{AUROC}(g_i(H_\theta^{(k)}), \mathbb{1}[y_i = 2])$ over fold k
- 7: **end for**
- 8: $\text{score}_\theta \leftarrow \frac{1}{K} \sum_k \text{score}_k$
- 9: **end for**
- 10: $\theta^* \leftarrow \arg \max_\theta \text{score}_\theta$
- 11: fit W_{θ^*} and train H_{θ^*} on all of $\mathcal{D}_{\text{train}}$
- 12: report $\text{Acc}(\lambda)$, escalation rate, and AUROCs of H_{θ^*} on $\mathcal{D}_{\text{test}}$

3.5 TRAINING THE DECISION HEAD

Self-labelling. The slow path is a deterministic function of the frozen backbone, so its correctness on a labelled question is a fixed attribute of that question. Given a set of questions with gold answers, we run the fast path (recording $h_i^{(\ell)}$ for a few candidate layers ℓ and the statistics s_i) and the slow path (recording a_i^{slow} and T_i), compare both with gold, and obtain the four-way label $y_i \in \{0, 1, 2, 3\}$ (Algorithm 1). Gold answers are compared locally after both passes and never enter a prompt. Nobody labels “should this question be escalated”.

Objective and regularisation. The head is trained by class-weighted cross-entropy on y_i with AdamW (Loshchilov & Hutter, 2019), weight decay 0.05, dropout 0.3, and early stopping on a validation slice of the training set. Two ingredients were necessary. First, the hidden state is high-dimensional ($d = 2560$ for the 4B backbone) relative to the number of labelled questions (about a thousand), and an MLP on the raw standardised h memorises the training set: in our first run training loss fell to 0.002 while held-out routing quality dropped below a router that ignores h entirely. We therefore fit a whitened PCA basis W on the standardised training hidden states and feed the head only the top r components. Second, the choice of layer and of r must not see the test data. We hold out whole subject categories as the test set and select the feature configuration by K -fold cross-validation grouped by category on the remaining data, scoring each configuration by the routing AUROC: how well g_i ranks the questions on which thinking helps ($y_i = 2$) above the rest (Algorithm 2). Selection on a single validation split was unstable at this data size and chose configurations that did not transfer.

Candidates. Θ contains a *stats-only* head (no hidden state; this is the floor that h must beat to justify its cost), one raw- h head per collected layer, and one PCA-bottleneck head per collected layer.

Table 1: Held-out accuracy and cost on three unseen MMLU-Pro categories (346 questions), Qwen3.5-4B. Tokens are mean generated thinking tokens per question. “Gain kept” is the fraction of the fast→slow accuracy gain recovered.

Policy	λ	Escalation	Accuracy	Tokens/q	Gain kept
Fast only (System One)	–	0%	0.540	0	0%
Slow only (always think)	–	100%	0.740	994	100%
Random routing	–	53%	0.646	527	53%
Stats-only head	0	48%	0.682	477	71%
LitJev-S2 (layer 24, PCA-64)	0.1	35%	0.691	345	76%
LitJev-S2 (layer 24, PCA-64)	0	53%	0.717	526	89%
LitJev-S2 (layer 24, PCA-64)	−0.1	84%	0.728	830	94%

The stats-only head is itself a useful artefact: it needs no hidden states at inference and can be served by any implementation of the fast path.

Serving. The trained head is a single safetensors file that records the backbone identifier and revision, the prompt format, the layer, and the PCA rank; the server refuses a head whose metadata does not match the loaded backbone. The standard endpoint is unchanged. A second endpoint accepts the same request body plus an optional $\{\lambda, B\}$ block and returns the same response shape.

4 EXPERIMENTS

4.1 SETUP

Backbone. Qwen3.5-4B, a hybrid checkpoint interleaving Gated DeltaNet layers (Yang et al., 2024) with full attention (32 layers, $d = 2560$), served in bfloat16 through Hugging Face Transformers (Wolf et al., 2020) on one H100. Cache replication copies both key/value and recurrent states. We chose a small backbone so that the fast/slow gap is large and the experiment is cheap; the question of whether the gap and the routing signal persist at larger scale is left open.

Data. MMLU-Pro (Wang et al., 2024), test split, 1,500 questions sampled with stride 8 across all 14 subject categories, each with up to ten options. Three categories (chemistry, economics, psychology; 346 questions) are held out entirely as the test set; the remaining eleven (1,154 questions) form the training set and the $K = 4$ selection folds. Every question is a `choice` question. Ten questions share one request with a generic state, matching the deployed ten-question batch.

Slow path. Greedy decoding, thinking budget $B = 1024$ tokens. The backbone hit the budget on 90.3% of questions (mean 994 tokens), so slow-path accuracy here is a floor and the gap to the fast path an underestimate.

Head. Collected layers $\ell \in \{-1, 16, 24\}$ where -1 is the final normalised output. PCA rank $r = 64$, hidden width $w = 64$ for bottleneck heads and 256 for raw heads, 32 for the stats-only head. Probe training for selection runs 10 epochs; the final head 60 epochs with patience 6. All training is on CPU and takes under a minute per configuration.

Metrics. Accuracy on the held-out categories as a function of escalation rate, swept over λ ; AUROC of g_i for ranking questions where thinking helps (*routing AUROC*); AUROC of the confidence c_i for fast-answer correctness; and expected thinking tokens per question. Baselines: fast only, slow only, the random-routing line, and the stats-only head.

4.2 MAIN RESULT

Table 1 and Figure 2 give the main result. Thinking helps on 24.6% of held-out questions and hurts on 4.6%, so the ceiling for any router is 0.740 and the fast floor is 0.540. At $\lambda = 0$ the decision head escalates 53% of questions and reaches 0.717: 89% of the available gain for 53% of the token cost.

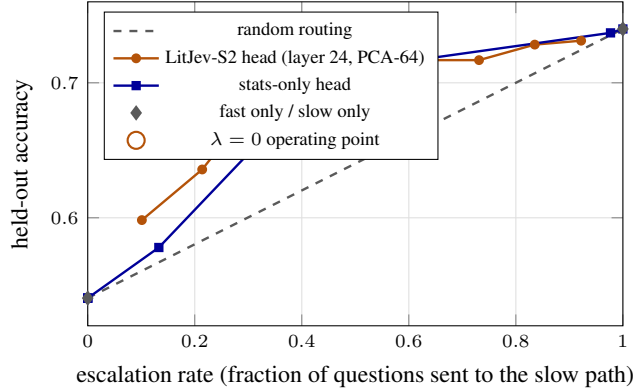


Figure 2: Coverage-accuracy curves on held-out categories. Each marker is one value of λ . The decision head dominates both the random-routing line and a router that sees only the fast answer distribution. No router at this AUROC reaches the slow-only point; the value is keeping most of the gain at a fraction of the cost.

Table 2: Feature configurations. *CV* columns are means over the four category folds of the training set and drive selection; *test* columns are on the held-out categories and are reported for all configurations only for analysis. Routing AUROC ranks questions where thinking helps; fast AUROC ranks fast-correct questions. The selected configuration is in bold. Non-parametric baselines on the test set: max probability 0.836, concentration 0.842 (fast AUROC).

Configuration	Input dim	CV (selection)		Test (held-out)	
		Routing	Fast	Routing	Fast
Stats only	8	0.671	0.757	0.704	0.838
Layer -1, raw	2568	0.700	0.748	0.677	0.824
Layer -1, PCA-64	72	0.694	0.753	0.648	0.843
Layer 16, raw	2568	0.701	0.666	0.644	0.738
Layer 16, PCA-64	72	0.668	0.729	0.716	0.837
Layer 24, raw	2568	0.729	0.748	0.718	0.824
Layer 24, PCA-64	72	0.758	0.766	0.721	0.834

At $\lambda = 0.1$ it keeps 76% of the gain for 35% of the cost. The curve lies above the random-routing line everywhere and above the stats-only head at every escalation rate, by about three accuracy points in the middle of the range (0.717 vs. 0.682 near 50% escalation; 0.691 vs. 0.656 near 33%).

4.3 WHERE THE ROUTING SIGNAL LIVES

Table 2 separates two questions. *Does the hidden state add anything beyond the fast distribution?* Yes, for routing: the selected configuration’s routing AUROC exceeds the stats-only head both in cross-validation (0.758 vs. 0.671) and on held-out categories (0.721 vs. 0.704), and this is what produces the three-point accuracy margin in Table 1. *Does it add anything for confidence?* No: fast-correct AUROC is 0.83–0.84 for every configuration and for the parameter-free max-probability baseline. The fast distribution already knows when the fast answer is wrong; what it does not know is whether thinking would fix it, and that is what the hidden state contributes.

Two further observations. Layer 24 of 32 carries a stronger routing signal than the final normalised output, consistent with probing results that place self-knowledge in intermediate layers (Azaria & Mitchell, 2023). And the PCA bottleneck is not optional at this data size: raw 2560-dimensional inputs score lower than their bottlenecked counterparts on the layer that matters and, in an earlier 1,000-question run, fell below the stats-only floor.

4.4 PER-CATEGORY BEHAVIOUR

Table 3 in the appendix breaks the full 1,500 questions down by subject. The fraction of questions where thinking helps ranges from 10% (biology) to 50% (math), and the fraction where it hurts from under 1% (physics) to 9% (engineering, where the slow path barely improves on the fast one). A good router must therefore be subject-sensitive, and the head sees no subject label; whatever it learns about subjects it learns from the hidden state. The held-out categories span this range (chemistry 35% helps, psychology 13%), so the reported numbers are not from an easy slice.

4.5 COST

At $\lambda = 0$ the routed policy generates 526 thinking tokens per question in expectation against 994 for always-thinking, and the decision itself is free: the head is a $72 \rightarrow 64 \rightarrow 4$ MLP evaluated on a vector the fast path already produces. The fast path’s own cost is unchanged at two forward calls and zero generated tokens. At the current 90% budget-hit rate the slow path is dominated by generation, so the token ratio is a faithful proxy for latency and GPU time.

5 LESSONS FROM FAILED RUNS

We report three failures because each is easy to reproduce and hard to diagnose from aggregate metrics.

The slow path that never thought. Our first implementation appended `<think>` to the fast prefix. The smoke test reported a thinking length of exactly one token on every question, and 1,000 questions were collected before the aggregate looked wrong. The cause is the empty thinking block that the chat template inserts when thinking is disabled: the model treats reasoning as already finished. The fix (Section 3.4) is to render the slow prompt with thinking enabled. Any pipeline that shares a prefix between a non-thinking and a thinking mode of a hybrid checkpoint should assert on thinking length.

Truncated reasoning underestimates the slow path. With $B = 512$, the backbone hit the budget on 98.5% of questions and slow-only accuracy was 0.616; with $B = 1024$ it is 0.740. In the 512 run the routed policy appeared to *exceed* always-thinking (0.642 vs. 0.616), which is impossible for a router with imperfect knowledge and was an artefact of truncation: escalation was selecting the questions the truncated slow path could still finish. We regard 1024 as a floor too.

Overfitting and unstable selection. Raw hidden states with ~ 750 training examples drove training loss to 0.002 and held-out routing AUROC below the stats-only head; selecting the layer on a single validation split then chose a raw configuration that did not transfer. Both the PCA bottleneck and grouped K -fold selection were introduced in response, and the selected configuration has agreed with the held-out ranking since.

6 LIMITATIONS

The evidence is from one backbone, one seed and one benchmark of `choice` questions. `score` and `noul` questions carry a type feature but were not present in training data. Held-out categories measure transfer across subjects within MMLU-Pro, not to other distributions; a deployment should collect records on its own question distribution, which Algorithm 1 makes cheap. Routing AUROC of 0.72 leaves substantial headroom, and we have not tested whether more labelled data, more layers, or a different bottleneck closes it. The slow path is greedy single-sample thinking with a fixed budget; a budget chosen per question, or sampling, would change both the labels and the ceiling. Finally, the head is trained by supervised learning on outcomes of a fixed slow path, so it cannot improve the slow path itself; that is a different problem.

7 CONCLUSION

A frozen typed-decision model can be taught, with a few hundred kilobytes of added parameters and no annotation beyond a labelled question set, to decide which questions are worth thinking about.

The decision reads a hidden state the model computes anyway, so it costs nothing on the fast path, and the cost of thinking is exposed as a single request-time threshold. On MMLU-Pro with a 4B backbone this keeps 89% of the accuracy that always-thinking buys for 53% of its tokens. The signal that makes this work is in an intermediate layer’s hidden state and is about *whether thinking would help*, not merely whether the fast answer is wrong; the latter the fast distribution already knows.

REFERENCES

- Pranjal Aggarwal and Sean Welleck. L1: Controlling how long a reasoning model thinks with reinforcement learning. *arXiv preprint arXiv:2503.04697*, 2025.
- Guillaume Alain and Yoshua Bengio. Understanding intermediate layers using linear classifier probes. In *International Conference on Learning Representations, Workshop Track*, 2017.
- Amos Azaria and Tom Mitchell. The internal state of an LLM knows when it’s lying. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023.
- Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023.
- Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems*, 2017.
- Alex Graves. Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*, 2016.
- Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *International Conference on Machine Learning*, 2017.
- Dan Hendrycks and Kevin Gimpel. A baseline for detecting misclassified and out-of-distribution examples in neural networks. In *International Conference on Learning Representations*, 2017.
- Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2021.
- Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, Scott Johnston, Sheer El-Showk, Andy Jones, Nelson Elhage, Tristan Hume, Anna Chen, Yuntao Bai, Sam Bowman, Stanislav Fort, Deep Ganguli, Danny Hernandez, Josh Jacobson, Jackson Kernion, Shauna Kravec, Liane Lovitt, Kamal Ndousse, Catherine Olsson, Sam Ringer, Dario Amodei, Tom Brown, Jack Clark, Nicholas Joseph, Ben Mann, Sam McCandlish, Chris Olah, and Jared Kaplan. Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221*, 2022.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.
- Mapika. decider: one-pass typed decisions with calibrated probabilities. <https://github.com/Mapika/decider>, 2026.
- Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. s1: Simple test-time scaling. *arXiv preprint arXiv:2501.19393*, 2025.
- Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs with preference data. *arXiv preprint arXiv:2406.18665*, 2024.
- Tal Schuster, Adam Fisch, Jai Gupta, Mostafa Dehghani, Dara Bahri, Vinh Q. Tran, Yi Tay, and Donald Metzler. Confident adaptive language modeling. In *Advances in Neural Information Processing Systems*, 2022.
- Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.

TypeSafe AI. Introducing System One models and Jev. <https://typesafe.ai/blog/introducing-system-one-models-and-jev>, 2026. Accessed 2026-09-18.

Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhramil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyan Jiang, Tianle Li, Max Ku, Kai Wang, Alex Zhuang, Rongqi Fan, Xiang Yue, and Wenhui Chen. MMLU-Pro: A more robust and challenging multi-task language understanding benchmark. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2024.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2020.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving Mamba2 with delta rule. *arXiv preprint arXiv:2412.06464*, 2024.

Eric Zhang. openjev-sglang: a Jev-compatible public API on open models. <https://github.com/ekzhang/openjev-sglang>, 2026.

Jiajie Zhang, Nianyi Lin, Lei Hou, Ling Feng, and Juanzi Li. AdaptThink: Reasoning models can learn when to think. *arXiv preprint arXiv:2505.13417*, 2025.

zhizh. openjev: local bilingual probability decisions. <https://github.com/zhizh/openjev>, 2026.

A PER-CATEGORY STATISTICS

Table 3: Fast and slow accuracy per MMLU-Pro category on all 1,500 collected questions, Qwen3.5-4B, $B = 1024$. *Helps*: fast wrong and slow right. *Hurts*: fast right and slow wrong. Starred categories are the held-out test set.

Category	n	Fast	Slow	Helps	Hurts
Biology	90	0.767	0.844	0.100	0.022
Business	99	0.374	0.778	0.444	0.040
Chemistry*	142	0.324	0.655	0.352	0.021
Computer science	51	0.647	0.765	0.157	0.039
Economics*	105	0.657	0.819	0.210	0.048
Engineering	117	0.470	0.513	0.137	0.094
Health	102	0.627	0.794	0.186	0.020
History	47	0.532	0.702	0.191	0.021
Law	138	0.355	0.507	0.225	0.072
Math	169	0.314	0.805	0.503	0.012
Other	116	0.543	0.647	0.155	0.052
Philosophy	62	0.355	0.677	0.339	0.016
Physics	163	0.380	0.791	0.417	0.006
Psychology*	99	0.727	0.778	0.131	0.081
All	1500	0.479	0.716	0.275	0.039

B PROMPT FORMATS

Fast path. System: “Evaluate the state using the question and labeled options that follow. Return only the option code. Do not explain or reason aloud.” User: the state. The chat template is

rendered with thinking disabled. Each branch suffix is `Question: {json}\nAnswer:`, where the JSON carries the question type, instructions, and the options as `{code, key, description}` triples with single-token codes.

Slow path. System: “*Evaluate the state using the question and labeled options. Think it through first, then answer with only the option code.*” User: the state, a blank line, and the same `Question: {json}` body. The template is rendered with thinking enabled, so it ends with `<think>`. After generation the sequence is closed with `\n</think>\n\nAnswer:` and read out at the final position.

C HYPERPARAMETERS

Fast-path temperature τ	1 (uncalibrated)
Distribution statistics	max prob, normalised entropy, top-2 margin, concentration, $\log K$, type one-hot
Collected layers	$\{-1, 16, 24\}$ (hidden_states indices)
PCA rank r	64, whitened, fit on standardised training hidden states
MLP width w	64 (PCA heads), 256 (raw heads), 32 (stats-only)
Optimiser	AdamW, lr 10^{-3} , weight decay 0.05, batch 256
Regularisation	dropout 0.3, class-weighted cross-entropy, early stopping (patience 6)
Selection	4 category-grouped folds, mean routing AUROC
Held-out categories	chemistry, economics, psychology
Slow-path decoding	greedy, $B = 1024$, stop at <code></think></code>
λ sweep	$\{-0.2, -0.1, -0.05, 0, 0.02, 0.05, 0.1, 0.2, 0.3, 0.5\}$